

Java Full Stack Development

1. PL/SQL





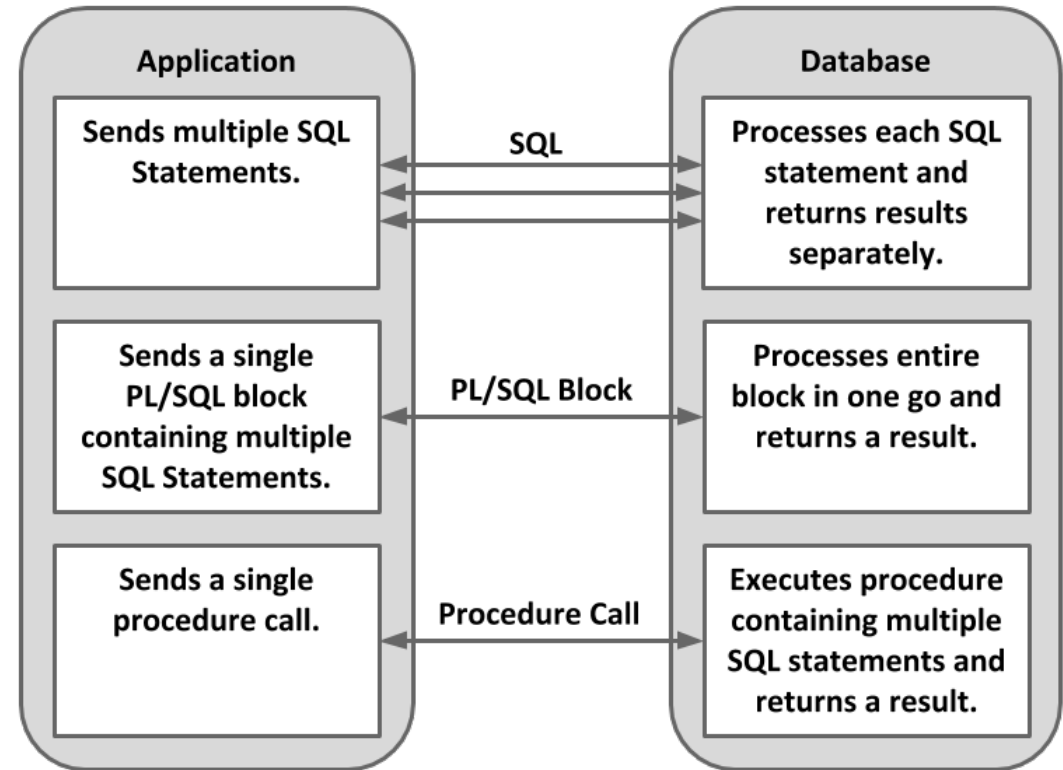
Module Topics

- PL/SQL
- Anonymous Blocks
- Stored Procedures
- Functions
- User Defined Types
- Packages
- Cursors
- Exceptions

PL/SQL

PL/SQL

- The problem SQL cannot solve in isolation
 - SQL is a set-based, declarative language
 - Operates on entire sets of rows at once and returns results
 - Has no control flow: no loops, no conditional branching, no variables, no error handling
 - Many real-world data operations require procedural logic
 - Validate a complex business rule
 - Perform a sequence of dependent operations, handle exceptions differently depending on what went wrong
 - Without a server-side procedural language
 - All of this logic must live in the application tier



PL/SQL

- What PL/SQL provides
 - A procedural extension to SQL that runs inside the Oracle database engine
 - Full control flow
 - IF/THEN/ELSE, CASE, LOOP, FOR, WHILE, GOTO
 - Variables, constants, and user-defined types
 - Exception handling
 - Named exceptions, user-defined exceptions, and propagation rules
 - Tight integration with SQL
 - SQL statements are first-class citizens inside PL/SQL blocks

PL/SQL Block

```
-- available online in file 'sample1'
DECLARE
  x NUMBER := 100;
BEGIN
  FOR i IN 1..10 LOOP
    IF MOD(i,2) = 0 THEN      -- i is even
      INSERT INTO temp VALUES (i, x, 'i is even');
    ELSE
      INSERT INTO temp VALUES (i, x, 'i is odd');
    END IF;
    x := x + 100;
  END LOOP;
  COMMIT;
END;
```

Output Table

```
SQL> SELECT * FROM temp ORDER BY col1;
```

NUM_COL1	NUM_COL2	CHAR_COL
1	100	i is odd
2	200	i is even
3	300	i is odd
4	400	i is even
5	500	i is odd
6	600	i is even
7	700	i is odd
8	800	i is even
9	900	i is odd
10	1000	i is even

PL/SQL

- The architectural motivation
 - Logic that runs in the database engine operates directly on the data without network round-trips
 - A complex operation requiring ten separate SQL calls from an application layer can be expressed as a single PL/SQL block
 - One network call, ten SQL operations
 - At enterprise data volumes, the reduction in network latency and round-trips is significant
 - The database engine can optimize the execution of SQL within PL/SQL using the same cost-based optimizer that optimizes standalone SQL

PL/SQL Block

```
-- available online in file 'sample1'
DECLARE
  x NUMBER := 100;
BEGIN
  FOR i IN 1..10 LOOP
    IF MOD(i,2) = 0 THEN      -- i is even
      INSERT INTO temp VALUES (i, x, 'i is even');
    ELSE
      INSERT INTO temp VALUES (i, x, 'i is odd');
    END IF;
    x := x + 100;
  END LOOP;
  COMMIT;
END;
```

Output Table

```
SQL> SELECT * FROM temp ORDER BY col1;
```

NUM_COL1	NUM_COL2	CHAR_COL
1	100	i is odd
2	200	i is even
3	300	i is odd
4	400	i is even
5	500	i is odd
6	600	i is even
7	700	i is odd
8	800	i is even
9	900	i is odd
10	1000	i is even

PL/SQL

- PL/SQL and Oracle architecture
 - PL/SQL blocks are compiled into an intermediate bytecode format and stored in the database
 - The PL/SQL engine and the SQL engine are separate but tightly integrated within Oracle
 - When a PL/SQL block executes a SQL statement
 - Control passes to the SQL engine, which executes it and returns the result to the PL/SQL engine
 - Called a context switch and has a small but measurable cost
 - Minimizing unnecessary context switches improves performance

PL/SQL Block

```
-- available online in file 'sample2'
DECLARE
  CURSOR c1 is
    SELECT ename, empno, sal FROM emp
    ORDER BY sal DESC;  -- start with highest paid employee
  my_ename VARCHAR2(10);
  my_empno NUMBER(4);
  my_sal    NUMBER(7,2);
BEGIN
  OPEN c1;
  FOR i IN 1..5 LOOP
    FETCH c1 INTO my_ename, my_empno, my_sal;
    EXIT WHEN c1%NOTFOUND;  /* in case the number requested */
                           /* is more than the total      */
                           /* number of employees      */
    INSERT INTO temp VALUES (my_sal, my_empno, my_ename);
    COMMIT;
  END LOOP;
  CLOSE c1;
END;
```

Output Table

```
SQL> SELECT * FROM temp ORDER BY col1 DESC;
```

NUM_COL1	NUM_COL2	CHAR_COL
5000	7839	KING
3000	7902	FORD
3000	7788	SCOTT
2975	7566	JONES
2850	7698	BLAKE

PL/SQL Block Structure

- Fundamental unit of PL/SQL code
 - Every PL/SQL program is a block
 - A block is the basic structural unit of PL/SQL
 - All PL/SQL code is built from blocks
 - Anonymous blocks, procedures, functions, triggers, packages, etc
 - A block has three sections
 - Declaration
 - Execution
 - Exception handling
 - Only the execution section is mandatory
 - The other two are optional

```
DECLARE
    -- Optional: variables, constants, cursors, user-defined types
    v_employee_count NUMBER;
    v_department_name VARCHAR2(100) := 'Engineering';
    c_max_salary      CONSTANT NUMBER := 250000;

BEGIN
    -- Mandatory: executable statements, SQL, control flow
    SELECT COUNT(*)
    INTO   v_employee_count
    FROM   employees
    WHERE  department_name = v_department_name;

    IF v_employee_count > 100 THEN
        DBMS_OUTPUT.PUT_LINE('Large department: ' || v_employee_count || ' employees');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Standard department: ' || v_employee_count || ' employees');
    END IF;

EXCEPTION
    -- Optional: handlers for named and unnamed exceptions
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Department not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);

END;
```

PL/SQL Block Structure

- Key structural rules
 - The DECLARE section introduces all variables used in the block
 - PL/SQL is statically typed and variables must be declared before use
 - The BEGIN/END section contains the executable logic
 - SQL statements, control flow, and calls to other procedures
 - The EXCEPTION section catches errors by name
 - Oracle provides a set of predefined exception names, and additional exceptions can be declared and raised by user code
 - Blocks can be nested
 - A BEGIN/END section can contain another complete block, which has its own optional DECLARE and EXCEPTION sections

```
DECLARE
    -- Optional: variables, constants, cursors, user-defined types
    v_employee_count    NUMBER;
    v_department_name    VARCHAR2(100) := 'Engineering';
    c_max_salary        CONSTANT NUMBER := 250000;

BEGIN
    -- Mandatory: executable statements, SQL, control flow
    SELECT COUNT(*)
    INTO    v_employee_count
    FROM    employees
    WHERE   department_name = v_department_name;

    IF v_employee_count > 100 THEN
        DBMS_OUTPUT.PUT_LINE('Large department: ' || v_employee_count || ' employees');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Standard department: ' || v_employee_count || ' employees');
    END IF;

EXCEPTION
    -- Optional: handlers for named and unnamed exceptions
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Department not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);

END;
```

Variable Typing

- The problem with hard-coded data types
 - PL/SQL variables must be declared with a data type before use
 - The obvious approach is to repeat the column's data type in the declaration
 - This works until the column definition changes
 - If full_name is widened from VARCHAR2(200) to VARCHAR2(400)
 - Every PL/SQL variable that hard-coded VARCHAR2(200) is now mismatched
 - In a large codebase, tracking down every affected declaration is tedious and error-prone`

```
DECLARE
  v_employee_id  NUMBER;
  v_full_name    VARCHAR2(200);
  v_salary       NUMBER(10,2);
  v_hire_date    DATE;
```

Variable Typing

- %TYPE
 - Anchors a variable to a single column
 - %TYPE declares a variable to have exactly the same data type as a specified table column
 - Oracle resolves the type at compile time by looking up the column definition in the data dictionary
 - If the column definition changes, the variable automatically picks up the new type the next time the PL/SQL unit is recompiled

```
DECLARE
-- These variables automatically match their column definitions
v_employee_id employees.employee_id%TYPE; -- whatever type employee_id is
v_full_name employees.full_name%TYPE; -- whatever type full_name is
v_salary employees.salary%TYPE; -- whatever type salary is
v_hire_date employees.hire_date%TYPE; -- whatever type hire_date is

BEGIN
SELECT employee_id, full_name, salary, hire_date
INTO v_employee_id, v_full_name, v_salary, v_hire_date
FROM employees
WHERE employee_id = 1001;

DBMS_OUTPUT.PUT_LINE(v_full_name || ' earns ' || v_salary);
END;
/
```


Variable Typing

- %ROWTYPE
 - Anchors a variable to an entire row
 - %ROWTYPE declares a single variable that holds all the columns of a table row
 - Individual columns are accessed using dot notation
 - Eliminates the need to declare one variable per column for single-row fetch operations

```
DECLARE
    v_emp employees%ROWTYPE;  -- one variable holds the entire row
BEGIN
    SELECT *
    INTO   v_emp
    FROM   employees
    WHERE  employee_id = 1001;

    -- Access individual columns via dot notation
    DBMS_OUTPUT.PUT_LINE('Name:      ' || v_emp.full_name);
    DBMS_OUTPUT.PUT_LINE('Salary:    ' || v_emp.salary);
    DBMS_OUTPUT.PUT_LINE('Hired:     ' || v_emp.hire_date);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_emp.department_id);
END;
/
```

Anonymous Blocks

Anonymous Blocks

- Anonymous block
 - A PL/SQL block submitted directly to Oracle without being stored as a named database object
 - It executes immediately and its compiled form is discarded after execution
 - Has no name, cannot be called by other code, cannot accept parameters
 - Executed from SQL*Plus, SQL Developer, application code, or a script

```
DECLARE
v_updated_count  NUMBER := 0;
  v_error_count   NUMBER := 0;
BEGIN
  -- Correct records where status was set incorrectly during a bulk load
  FOR rec IN (
    SELECT employee_id
    FROM   employees
    WHERE  status = 'ACTIVE'  -- typo introduced during migration
  ) LOOP
    BEGIN
      UPDATE employees
      SET   status = 'ACTIVE'
      WHERE employee_id = rec.employee_id;
      v_updated_count := v_updated_count + 1;
    EXCEPTION
      WHEN OTHERS THEN
        v_error_count := v_error_count + 1;
        DBMS_OUTPUT.PUT_LINE('Failed on employee_id: ' ||
                              rec.employee_id);
    END;
  END LOOP;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Updated: ' || v_updated_count || ' rows');
  DBMS_OUTPUT.PUT_LINE('Errors:  ' || v_error_count  || ' rows');
END;
/
```

Anonymous Blocks

- When anonymous blocks are appropriate
 - One-time administrative operations
 - Data migrations, batch corrections, environment setup scripts
 - Testing and development
 - Exploring PL/SQL logic before formalizing it as a stored procedure
 - Deployment scripts
 - Executing a complex sequence of DDL and DML as part of a release
 - Interactive exploration in a development session

```
DECLARE
v_updated_count  NUMBER := 0;
  v_error_count   NUMBER := 0;
BEGIN
  -- Correct records where status was set incorrectly during a bulk load
  FOR rec IN (
    SELECT employee_id
    FROM   employees
    WHERE  status = 'ACTIVE'  -- typo introduced during migration
  ) LOOP
    BEGIN
      UPDATE employees
      SET   status = 'ACTIVE'
      WHERE employee_id = rec.employee_id;
      v_updated_count := v_updated_count + 1;
    EXCEPTION
      WHEN OTHERS THEN
        v_error_count := v_error_count + 1;
        DBMS_OUTPUT.PUT_LINE('Failed on employee_id: ' ||
                              rec.employee_id);
    END;
  END LOOP;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Updated: ' || v_updated_count || ' rows');
  DBMS_OUTPUT.PUT_LINE('Errors:  ' || v_error_count  || ' rows');
END;
/
```

Anonymous Blocks

- When anonymous blocks are not appropriate
 - Logic that needs to be called repeatedly from application code
 - Use a stored procedure
 - Logic that must be shared across multiple callers
 - Use a stored procedure or package
 - Logic that must be secured with specific EXECUTE privileges
 - Anonymous blocks inherit the caller's privileges directly

```
DECLARE
v_updated_count  NUMBER := 0;
v_error_count    NUMBER := 0;
BEGIN
    -- Correct records where status was set incorrectly during a bulk load
    FOR rec IN (
        SELECT employee_id
        FROM   employees
        WHERE  status = 'ACTIVE' -- typo introduced during migration
    ) LOOP
        BEGIN
            UPDATE employees
            SET   status = 'ACTIVE'
            WHERE employee_id = rec.employee_id;
            v_updated_count := v_updated_count + 1;
        EXCEPTION
            WHEN OTHERS THEN
                v_error_count := v_error_count + 1;
                DBMS_OUTPUT.PUT_LINE('Failed on employee_id: ' ||
                                     rec.employee_id);
        END;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Updated: ' || v_updated_count || ' rows');
    DBMS_OUTPUT.PUT_LINE('Errors: ' || v_error_count || ' rows');
END;
/
```


Anonymous Blocks

- PL/SQL equivalent of a shell script
- Useful for one-time execution
 - Not appropriate as a permanent part of an application

```
DECLARE
v_updated_count  NUMBER := 0;
  v_error_count   NUMBER := 0;
BEGIN
  -- Correct records where status was set incorrectly during a bulk load
  FOR rec IN (
    SELECT employee_id
    FROM   employees
    WHERE  status = 'ACTIVE'  -- typo introduced during migration
  ) LOOP
  BEGIN
    UPDATE employees
    SET   status = 'ACTIVE'
    WHERE employee_id = rec.employee_id;
    v_updated_count := v_updated_count + 1;
  EXCEPTION
    WHEN OTHERS THEN
      v_error_count := v_error_count + 1;
      DBMS_OUTPUT.PUT_LINE('Failed on employee_id: ' ||
        rec.employee_id);
  END;
END LOOP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Updated: ' || v_updated_count || ' rows');
DBMS_OUTPUT.PUT_LINE('Errors: ' || v_error_count || ' rows');
END;
/
```

Anonymous Blocks

- The example demonstrates
 - Using a cursor FOR loop
 - No explicit OPEN, FETCH, CLOSE required)
 - Nesting a BEGIN/EXCEPTION/END block inside the loop to handle errors per-row without aborting the entire batch
 - Maintaining counters, and committing only after all work is done.
 - Each row might fail independently
 - Failures should be logged rather than causing the entire batch to roll back
 - For high-volume operations where all rows should succeed or all should fail, a set-based UPDATE without a loop is far more efficient
 - The choice between row-by-row and set-based operations is a PL/SQL design decision

```
DECLARE
v_updated_count  NUMBER := 0;
  v_error_count   NUMBER := 0;
BEGIN
  -- Correct records where status was set incorrectly during a bulk load
  FOR rec IN (
    SELECT employee_id
    FROM   employees
    WHERE  status = 'ACTIVE'  -- typo introduced during migration
  ) LOOP
    BEGIN
      UPDATE employees
      SET   status = 'ACTIVE'
      WHERE employee_id = rec.employee_id;
      v_updated_count := v_updated_count + 1;
    EXCEPTION
      WHEN OTHERS THEN
        v_error_count := v_error_count + 1;
        DBMS_OUTPUT.PUT_LINE('Failed on employee_id: ' ||
                              rec.employee_id);
    END;
  END LOOP;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Updated: ' || v_updated_count || ' rows');
  DBMS_OUTPUT.PUT_LINE('Errors:  ' || v_error_count  || ' rows');
END;
/
```

Stored Procedures

Stored Procedure

- Stored procedure
 - A named PL/SQL block compiled and stored permanently in the database as a schema object
 - Can accept input parameters, produce output parameters, or both
 - Called by name from application code, other PL/SQL blocks, or SQL*Plus
 - Subject to Oracle's privilege model
 - EXECUTE privilege controls who can call it

```
CREATE OR REPLACE PROCEDURE transfer_employee (  
    p_employee_id    IN    employees.employee_id%TYPE,  
    p_new_dept_id    IN    departments.department_id%TYPE,  
    p_effective_date IN    DATE DEFAULT SYSDATE,  
    p_result_code    OUT   VARCHAR2  
) AS  
    v_current_dept    employees.department_id%TYPE;  
    v_dept_exists     NUMBER;  
BEGIN  
    -- Validate the target department exists  
    SELECT COUNT(*) INTO v_dept_exists  
    FROM    departments  
    WHERE   department_id = p_new_dept_id;  
  
    IF v_dept_exists = 0 THEN  
        p_result_code := 'ERR_DEPT_NOT_FOUND';  
        RETURN;  
    END IF;
```

Stored Procedure

- Stored procedures architecturally
 - Encapsulate complex logic in one place
 - Callers do not need to know how the logic works, only what parameters to pass
 - Reduce network traffic
 - A single EXECUTE call replaces multiple SQL round-trips

```
-- Capture current department for audit
SELECT department_id INTO v_current_dept
FROM   employees
WHERE  employee_id = p_employee_id;

-- Perform the transfer
UPDATE employees
SET    department_id    = p_new_dept_id,
       last_modified    = p_effective_date
WHERE  employee_id      = p_employee_id;

-- Write audit record
INSERT INTO employee_audit_log (
    employee_id, action, old_dept_id, new_dept_id, action_date
) VALUES (
    employee_id p_employee_id, action 'TRANSFER',
    old_dept_id v_current_dept, new_dept_id p_new_dept_id,
    action_date p_effective_date
);

COMMIT;
p_result_code := 'SUCCESS';
```

Stored Procedure

- Stored procedures architecturally
 - Can be granted EXECUTE privilege independently of table access
 - The application can be granted permission to call a procedure without being granted direct SELECT, INSERT, UPDATE, or DELETE on the underlying tables
 - Changes to the implementation do not require changes to the caller
 - As long as the parameter signature remains stable

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_result_code := 'ERR_EMPLOYEE_NOT_FOUND';
ROLLBACK;
WHEN OTHERS THEN
    p_result_code := 'ERR_UNEXPECTED: ' || SQLERRM;
ROLLBACK;
END transfer_employee;
/
```

Parameter modes

Mode	Direction	Description
IN	Caller to procedure	Read-only inside the procedure; value supplied by the caller
OUT	Procedure to caller	Write-only inside the procedure; value returned to the caller
IN OUT	Both directions	Caller supplies a value; procedure may modify and return it

Stored Procedure

- CREATE OR REPLACE

- Standard way to create or update a stored procedure
- If the procedure does not exist it is created; if it already exists it is replaced atomically
 - Does not require dropping the procedure first, which would remove any grants on it

- RETURN statement

- Inside a procedure, exits the procedure immediately without raising an exception
- Handles the validation failure gracefully
- Set the output parameter and return, rather than raising an exception that the caller must catch

```
CREATE OR REPLACE PROCEDURE transfer_employee (  
    p_employee_id    IN    employees.employee_id%TYPE,  
    p_new_dept_id    IN    departments.department_id%TYPE,  
    p_effective_date IN    DATE DEFAULT SYSDATE,  
    p_result_code    OUT  VARCHAR2  
) AS  
    v_current_dept    employees.department_id%TYPE;  
    v_dept_exists     NUMBER;  
BEGIN  
    -- Validate the target department exists  
    SELECT COUNT(*) INTO v_dept_exists  
    FROM    departments  
    WHERE   department_id = p_new_dept_id;  
  
    IF v_dept_exists = 0 THEN  
        p_result_code := 'ERR_DEPT_NOT_FOUND';  
        RETURN;  
    END IF;
```

Stored Procedure

- COMMIT inside a procedure
 - Design choice
 - One option is that stored procedures never commit, leaving transaction control to the caller
 - Or prefer that a procedure encapsulates a complete unit of work including its commit
 - Both are valid options, but the design choices team must be consistent
 - Privilege model
 - Granting EXECUTE on a procedure rather than direct table access
 - Common security pattern in enterprise Oracle systems
 - The procedure acts as an API
 - Callers can only perform the operations the procedure exposes, with the parameters and validations the procedure enforces
 - And cannot directly manipulate the underlying tables.

```
-- Capture current department for audit
SELECT department_id INTO v_current_dept
FROM   employees
WHERE  employee_id = p_employee_id;

-- Perform the transfer
UPDATE employees
SET    department_id = p_new_dept_id,
       last_modified  = p_effective_date
WHERE  employee_id   = p_employee_id;

-- Write audit record
INSERT INTO employee_audit_log (
    employee_id, action, old_dept_id, new_dept_id, action_date
) VALUES (
    employee_id p_employee_id, action 'TRANSFER',
    old_dept_id v_current_dept, new_dept_id p_new_dept_id,
    action_date p_effective_date
);

COMMIT;
p_result_code := 'SUCCESS';
```

Functions

Functions

- Differences from a procedure
 - A function is a named PL/SQL block that returns exactly one value to its caller
 - It is declared with a RETURN type and must execute a RETURN statement that supplies the value
 - Functions can be called from SQL statements as well as from PL/SQL
 - This cannot be done with procedures
 - Functions called from SQL must follow specific rules to be safe for use in queries

```
CREATE OR REPLACE FUNCTION get_employee_grade (  
    p_salary IN employees.salary%TYPE  
) RETURN VARCHAR2 AS  
BEGIN  
    RETURN CASE  
        WHEN p_salary >= 200000 THEN 'EXECUTIVE'  
        WHEN p_salary >= 120000 THEN 'SENIOR'  
        WHEN p_salary >= 70000  THEN 'MID'  
        ELSE                       'JUNIOR'  
    END;  
END get_employee_grade;  
/  
  
-- Calling from SQL -- this is what makes functions powerful  
SELECT employee_id,  
       full_name,  
       salary,  
       get_employee_grade(salary) AS grade  
FROM   employees  
WHERE  get_employee_grade(salary) = 'SENIOR';
```

Functions

- Functions versus procedures
 - Use a function when the operation produces a single return value that will be used in a SQL expression or passed directly to another operation
 - Use a procedure when the operation performs an action, produces multiple output values, or needs to control transactions
 - Avoid complex functions in WHERE clauses on large tables
 - Oracle calls the function once per row evaluated, which adds CPU overhead and may prevent index use

```
CREATE OR REPLACE FUNCTION get_employee_grade (  
    p_salary IN employees.salary%TYPE  
) RETURN VARCHAR2 AS  
BEGIN  
    RETURN CASE  
        WHEN p_salary >= 200000 THEN 'EXECUTIVE'  
        WHEN p_salary >= 120000 THEN 'SENIOR'  
        WHEN p_salary >= 70000  THEN 'MID'  
        ELSE                       'JUNIOR'  
    END;  
END get_employee_grade;  
/  
  
-- Calling from SQL -- this is what makes functions powerful  
SELECT employee_id,  
       full_name,  
       salary,  
       get_employee_grade(salary) AS grade  
FROM   employees  
WHERE  get_employee_grade(salary) = 'SENIOR';
```

Functions

- Determinism and SQL safety
 - A function called from SQL must not modify database state
 - No INSERT, UPDATE, DELETE, or DDL inside a function used in a query
 - A DETERMINISTIC function always returns the same output for the same input
 - Oracle can cache the result and avoid repeated calls for the same value
 - Declaring a function DETERMINISTIC when it genuinely is allows Oracle to optimize queries that call it

```
CREATE OR REPLACE FUNCTION calculate_annual_salary (  
    p_monthly_salary IN NUMBER  
) RETURN NUMBER DETERMINISTIC AS  
BEGIN  
    RETURN p_monthly_salary * 12;  
END;  
/
```

Functions

- The DETERMINISTIC keyword is a contract
 - If a function is declared DETERMINISTIC but actually returns different values for the same input
 - For example, because it reads from a table that changes
 - Query results can be wrong
 - Because Oracle may use a cached result from an earlier call instead of re-evaluating

```
CREATE OR REPLACE FUNCTION calculate_annual_salary (  
    p_monthly_salary IN NUMBER  
) RETURN NUMBER DETERMINISTIC AS  
BEGIN  
    RETURN p_monthly_salary * 12;  
END;  
/
```


Functions

- Functions

- Can appear anywhere in a SQL statement where a value is expected
 - In the SELECT list, in a WHERE clause, in an ORDER BY, in a GROUP BY, or as an argument to another function
- Useful for encapsulating complex business logic that needs to be applied row by row in queries
 - A function in a WHERE clause is evaluated for every row Oracle considers
 - If the query accesses a million rows before filtering, the function is called a million times

```
CREATE OR REPLACE FUNCTION get_employee_grade (  
    p_salary IN employees.salary%TYPE  
) RETURN VARCHAR2 AS  
BEGIN  
    RETURN CASE  
        WHEN p_salary >= 200000 THEN 'EXECUTIVE'  
        WHEN p_salary >= 120000 THEN 'SENIOR'  
        WHEN p_salary >= 70000  THEN 'MID'  
        ELSE                       'JUNIOR'  
    END;  
END get_employee_grade;  
/  
  
-- Calling from SQL -- this is what makes functions powerful  
SELECT employee_id,  
       full_name,  
       salary,  
       get_employee_grade(salary) AS grade  
FROM   employees  
WHERE  get_employee_grade(salary) = 'SENIOR';
```

Functions

- Functions and Indexes

- A function in a WHERE clause is evaluated for every row Oracle considers
- Prevents Oracle from using an index on the underlying column
 - Because the function transforms the value before comparison
- In the example
 - Oracle has no choice but to read every row in the table, apply UPPER() to the last_name value of each one
 - Then check whether the result equals 'SMITH'
 - The index may be present but completely useless for this query because Oracle ignores it entirely

```
-- The table has an index on last_name
-- The index stores: 'Jones', 'Smith', 'Williams', etc.

-- This predicate CAN use the index
-- Oracle looks up 'Smith' directly in the index
WHERE last_name = 'Smith'

-- This predicate CANNOT use the index
-- Oracle needs to know which rows produce 'SMITH' when UPPER() is applied
-- The index stores 'Smith', not 'SMITH' -- it cannot answer this question
WHERE UPPER(last_name) = 'SMITH'
```

Functions

- The solution
 - When a function-based filter is needed at scale
 - Create a function-based index that pre-computes the function result
- The general principle is
 - The expression in the WHERE clause and the expression in the index must match exactly
 - If the index is on the raw column and the predicate applies a function, they do not match and the index is bypassed
 - If the index is on the function-applied value and the predicate uses the same function, they match and the index can be used.

```
-- This index stores: 'JONES', 'SMITH', 'WILLIAMS', etc.  
CREATE INDEX idx_emp_upper_name ON employees (UPPER(last_name));  
  
-- Now this predicate CAN use the index  
-- Oracle looks up 'SMITH' in an index that actually stores 'SMITH'  
WHERE UPPER(last_name) = 'SMITH'
```

User Defined Types

User Defined Types

- Why types exist
 - PL/SQL's built-in scalar types (NUMBER, VARCHAR2, DATE) describe single values
 - Real programs need to group related values together, and work with sets of rows as a unit
 - PL/SQL provides three categories of user-defined type for this purpose
 - Records
 - Collections
 - Object types
 - Records and collections are most commonly used in application development

```
-- Declaring a RECORD type
DECLARE
    TYPE t_employee_summary IS RECORD (
        employee_id    employees.employee_id%TYPE,
        full_name       employees.full_name%TYPE,
        department      departments.department_name%TYPE,
        grade           VARCHAR2(20),
        years_service   NUMBER
    );

    v_emp t_employee_summary; -- a variable of that type

BEGIN
    -- Assign fields individually
    v_emp.employee_id := 1001;
    v_emp.full_name    := 'Jane Smith';
    v_emp.grade        := 'SENIOR';
    v_emp.years_service := 7;

    DBMS_OUTPUT.PUT_LINE(v_emp.full_name || ' -- Grade: ' || v_emp.grade);
END;
/
```

Record Types

- A RECORD is a composite type
 - A named set of fields, each with its own data type
 - Conceptually similar to a struct in other languages
 - Used to pass a group of related values between procedures without needing a separate parameter for each field
 - Note that %TYPE anchoring works inside RECORD definitions
 - Fields stay in sync with their source columns automatically

```
-- Declaring a RECORD type
DECLARE
    TYPE t_employee_summary IS RECORD (
        employee_id    employees.employee_id%TYPE,
        full_name       employees.full_name%TYPE,
        department      departments.department_name%TYPE,
        grade           VARCHAR2(20),
        years_service   NUMBER
    );

    v_emp t_employee_summary; -- a variable of that type

BEGIN
    -- Assign fields individually
    v_emp.employee_id := 1001;
    v_emp.full_name    := 'Jane Smith';
    v_emp.grade        := 'SENIOR';
    v_emp.years_service := 7;

    DBMS_OUTPUT.PUT_LINE(v_emp.full_name || ' -- Grade: ' || v_emp.grade);
END;
/
```

Collection Types

- A COLLECTION
 - An ordered or keyed set of elements, all of the same type
 - Three collection types exist in PL/SQL
 - The collection types differ in how they are indexed and where they can be stored

Type	Index	Can be stored in a DB column	Notes
Associative array	INTEGER or VARCHAR2 -- chosen at declaration	No -- PL/SQL memory only	Most flexible; the go-to for in-memory lookup tables
Nested table	INTEGER, always starting at 1	Yes	Unbounded; elements can be deleted leaving gaps
VARRAY	INTEGER, always starting at 1	Yes	Bounded; maximum size declared upfront

Collection Types

- Associative array
 - Can be keyed by INTEGER or VARCHAR2
 - Exists only in PL/SQL memory
 - Not storable in a database column
 - Natural choice for in-memory lookup tables
- Common use case
 - When a procedure needs to look up a value repeatedly by a key value
 - In memory storage avoids context switching

```
DECLARE
    TYPE t_salary_map IS TABLE OF NUMBER
        INDEX BY VARCHAR2(100); -- key is a string (employee name)

    v_salaries t_salary_map;
BEGIN
    v_salaries('Jane Smith') := 95000;
    v_salaries('John Brown') := 82000;

    DBMS_OUTPUT.PUT_LINE('Jane earns: ' || v_salaries('Jane Smith'));
END;
/
```


Collection Types

- Nested table
 - A nested table type is used as if it was a column in a regular table
 - Oracle does not store the collection inline in the row
 - It stores the collection's elements in a separate storage table that Oracle manages automatically
 - Uses a hidden foreign key linking each element back to its parent row.
 - Not used in modern Oracle
 - Originally intended to support object-relational features
 - A properly normalized table with a lookup key is preferred

```
DECLARE
    TYPE t_name_list IS TABLE OF VARCHAR2(200);

    v_names t_name_list := t_name_list(); -- must be initialized
BEGIN
    v_names.EXTEND;
    v_names(1) := 'Jane Smith';
    v_names.EXTEND;
    v_names(2) := 'John Brown';

    -- Iterate using indices
    FOR i IN 1 .. v_names.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE(v_names(i));
    END LOOP;
END;
/
```

Collection Types

- Collections of records
 - Used with BULK COLLECT
 - The most practically important combination
 - A collection where each element is a record
 - Exactly what BULK COLLECT populates when fetching multiple rows from a query
 - Extensively used in PL/SQL

```
DECLARE
    TYPE t_employee_rec IS RECORD (
        employee_id employees.employee_id%TYPE,
        full_name    employees.full_name%TYPE,
        salary       employees.salary%TYPE
    );

    TYPE t_employee_list IS TABLE OF t_employee_rec;

    v_employees t_employee_list;

BEGIN
    -- Fetch all active employees into the collection in one context switch
    SELECT employee_id, full_name, salary
    BULK COLLECT INTO v_employees
    FROM employees
    WHERE status = 'ACTIVE';

    -- Process the collection entirely in PL/SQL -- no further SQL calls
    FOR i IN 1 .. v_employees.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE(
            v_employees(i).full_name || ': ' || v_employees(i).salary
        );
    END LOOP;
END;
```

Packages

Packages

- Module system of PL/SQL
 - Named container that groups related procedures, functions, variables, types, and cursors into a single schema object
- Two parts
 - A specification is the public interface
 - Declares what is publicly visible
 - The implementation body
 - Defines the operational code that executes
 - Code that calls a packaged procedure only needs access to the specification
 - The body is private

```
-- The specification: the public contract
CREATE OR REPLACE PACKAGE employee_mgmt AS

    -- Public type definition
    TYPE t_employee_record IS RECORD (
        employee_id employees.employee_id%TYPE,
        full_name    employees.full_name%TYPE,
        grade        VARCHAR2(20)
    );

    -- Public procedure declarations
    PROCEDURE hire_employee (
        p_full_name  IN  VARCHAR2,
        p_dept_id    IN  NUMBER,
        p_salary     IN  NUMBER,
        p_employee_id OUT NUMBER
    );

    PROCEDURE terminate_employee (
        p_employee_id IN NUMBER,
        p_reason       IN VARCHAR2
    );

    -- Public function declaration
    FUNCTION get_headcount (p_dept_id IN NUMBER) RETURN NUMBER;

END employee_mgmt;
/
```

Packages

- They are the correct unit of organization for any non-trivial PL/SQL codebase
 - Related logic lives together
 - All employee operations in one package
 - All audit operations in another
 - Package-level variables persist for the duration of a session
 - Useful for caching configuration values or tracking session state
 - Changing the body does not invalidate dependent objects as long as the specification is unchanged
 - A single EXECUTE grant on a package gives access to all public procedures and functions in it

```
-- The body: the implementation
CREATE OR REPLACE PACKAGE BODY employee_mgmt AS

    -- Private helper: not declared in the spec, not callable externally
    PROCEDURE write_audit (p_action IN VARCHAR2, p_employee_id IN NUMBER) AS
    BEGIN
        INSERT INTO employee_audit_log (action, employee_id, action_date)
        VALUES (p_action, p_employee_id, SYSDATE);
    END write_audit;

    PROCEDURE hire_employee (
        p_full_name  IN  VARCHAR2,
        p_dept_id    IN  NUMBER,
        p_salary     IN  NUMBER,
        p_employee_id OUT NUMBER
    ) AS
    BEGIN
        INSERT INTO employees (full_name, department_id, salary, hire_date, status)
        VALUES (p_full_name, p_dept_id, p_salary, SYSDATE, 'ACTIVE')
        RETURNING employee_id INTO p_employee_id;

        write_audit('HIRE', p_employee_id);
        COMMIT;
    END hire_employee;

    FUNCTION get_headcount (p_dept_id IN NUMBER) RETURN NUMBER AS
        v_count NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_count
        FROM   employees
        WHERE  department_id = p_dept_id
        AND    status        = 'ACTIVE';
        RETURN v_count;
    END get_headcount;

    -- terminate_employee implementation omitted for brevity

END employee_mgmt;
/
```

Packages

- Mental Model
 - PL/SQL equivalent to namespaces or Java packages
 - Introduces modularity into the codebase
 - Reduced coupling and higher cohesion
 - Separation of interface and implementation

```
-- Calling packaged code
BEGIN
    employee_mgmt.hire_employee(
        p_full_name  => 'Jane Smith',
        p_dept_id    => 10,
        p_salary     => 95000,
        p_employee_id => :new_id
    );
END;
/
```

Cursors

Cursors

- What a cursor is
 - Named pointer to the result set of a SQL query that allows PL/SQL code to process query results one row at a time
 - Oracle automatically manages a cursor for every SQL statement executed
- Implicit cursors
 - Created automatically by Oracle for every INSERT, UPDATE, DELETE, and single-row SELECT INTO
 - The developer does not declare them

```
-- Oracle manages OPEN, FETCH, and CLOSE automatically
-- The loop variable takes the type of each fetched row
BEGIN
    FOR emp_rec IN (
        SELECT employee_id, full_name, salary
        FROM   employees
        WHERE  department_id = 10
        AND    status        = 'ACTIVE'
        ORDER BY salary DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            emp_rec.full_name || ' -- ' || emp_rec.salary
        );
    END LOOP;
END;
/
```


Cursors

- Explicit cursors
 - Declared by the developer for queries expected to return multiple rows
- Performance consideration
 - Avoid row-by-row when possible
 - Every FETCH is a context switch between the PL/SQL and SQL engines
 - Processing one million rows one at a time generates one million context switches
 - BULK COLLECT fetches many rows in a single context switch
 - FORALL executes DML for many rows in a single context switch
 - Use BULK COLLECT and FORALL when processing large volumes of data in PL/SQL

```
DECLARE
    CURSOR c_employees (p_dept_id IN NUMBER) IS
        SELECT employee_id, full_name, salary
        FROM   employees
        WHERE  department_id = p_dept_id
        AND    status = 'ACTIVE';

    v_emp c_employees%ROWTYPE;
BEGIN
    OPEN c_employees(10);
    LOOP
        FETCH c_employees INTO v_emp;
        EXIT WHEN c_employees%NOTFOUND;
        -- process v_emp
    END LOOP;
    CLOSE c_employees;
END;
/
```

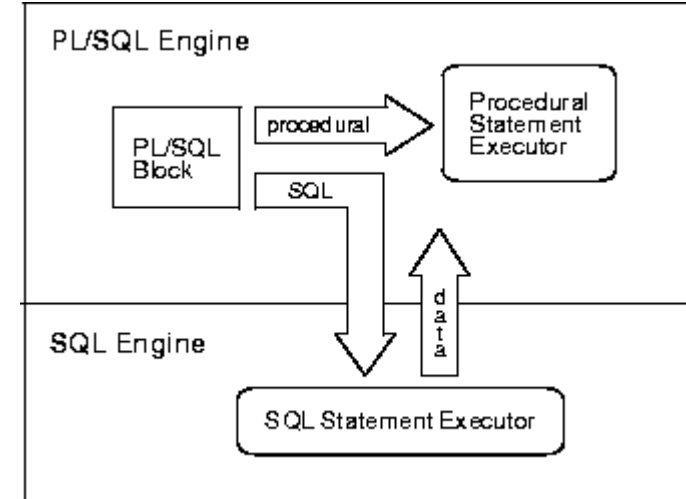
Cursors

- FOR loop
 - Should be the default pattern in almost all cases
 - Oracle handles the OPEN, FETCH, CLOSE lifecycle automatically
 - And the loop variable is implicitly typed to match the query's column list
 - Forgetting to close an explicit cursor is a common resource leak in PL/SQL code
 - The cursor FOR loop eliminates this class of bug entirely
- BULK COLLECT and FORALL
 - Solution to the "slow-by-slow" anti-pattern
 - BULK COLLECT fetches an entire result set or a specified number of rows into a PL/SQL collection in a single context switch
 - FORALL executes the same DML statement for every element of a collection in a single context switch, rather than one DML call per element

```
-- Oracle manages OPEN, FETCH, and CLOSE automatically
-- The loop variable takes the type of each fetched row
BEGIN
    FOR emp_rec IN (
        SELECT employee_id, full_name, salary
        FROM   employees
        WHERE  department_id = 10
        AND    status        = 'ACTIVE'
        ORDER BY salary DESC
    ) LOOP
        DBMS_OUTPUT.PUT_LINE(
            emp_rec.full_name || ' -- ' || emp_rec.salary
        );
    END LOOP;
END;
/
```

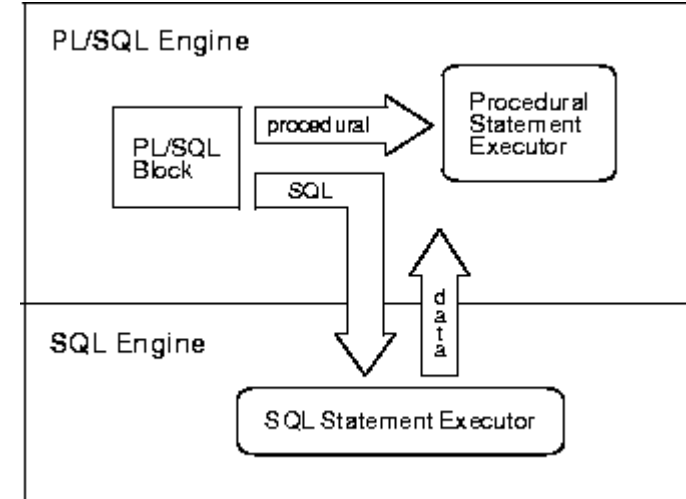
Context Switching

- When Oracle runs a PL/SQL block, two distinct engines are involved
 - The PL/SQL engine
 - Handles the procedural parts:
 - Variable assignments, IF statements, loops, calculations, exception handling
 - The SQL engine handles all data access
 - SELECT, INSERT, UPDATE, DELETE



Context Switching

- Costs
 - Each SWITCH involves
 - marshalling the SQL statement
 - Passing it to the SQL engine and, executing it
 - Marshalling the result, and returning it to the PL/SQL engine
 - For a single SQL call this overhead is negligible
 - But it doesn't scale well for large numbers of rows
 - BULK COLLECT and FORALL use batches
 - BULK COLLECT fetches thousands of rows in a single switch
 - FORALL sends the entire collection back to the SQL engine in a single switch for the DML



Exceptions

Exception Handling

- Importance
 - A PL/SQL block that does not handle exceptions propagates errors to its caller unmodified
 - In a multi-tier application, an unhandled Oracle exception surfaces as a raw ORA-error code
 - Not useful to an end user and potentially revealing internal schema details
 - Well-designed exception handling
 - Catches errors at the right level
 - Provides meaningful context
 - Maintains data integrity

```
DECLARE
    e_salary_too_high  EXCEPTION;
    c_max_salary       CONSTANT NUMBER := 500000;
BEGIN
    IF p_new_salary > c_max_salary THEN
        RAISE e_salary_too_high;
    END IF;

    UPDATE employees SET salary = p_new_salary WHERE employee_id = p_emp_id;

EXCEPTION
    WHEN e_salary_too_high THEN
        DBMS_OUTPUT.PUT_LINE('Salary exceeds maximum permitted value');
        ROLLBACK;
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Constraint violation on update');
        ROLLBACK;
    WHEN OTHERS THEN
        -- Always log; never silently swallow
        INSERT INTO error_log (error_code, error_message, logged_at)
        VALUES (SQLCODE, SQLERRM, SYSDATE);
        COMMIT;    -- commit the log entry even though the main transaction roll
        RAISE;     -- re-raise so the caller knows something went wrong

END;
/
```

Exception Handling

- EXCEPTION section
 - Sits between the end of the executable code and the END keyword of the block
 - It is not a separate block
 - It is a section within the same block
 - Once the EXCEPTION section handles the error
 - Control does not return to the BEGIN section
 - The block finishes after the handler completea
 - If the handler re-raises the exception with RAISE, the error propagates to the enclosing block (if one exists) or to the caller

```
BEGIN
    statement_1;  -- executes
    statement_2;  -- raises an error here
    statement_3;  -- never reached -- Oracle jumps to EXCEPTION immediately

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- control arrives here
END;
```

Exception Handling

- WHEN OTHERS handler is a catch-all
 - Matches every exception that was not caught by a more specific handler above it
 - Useful as a safety net
 - if Oracle arrived here, something went wrong that was not anticipated
 - The code must respond in a way that preserves the ability to diagnose what happened
 - Logging preserves the evidence
 - Re-raising tells the caller something went wrong
 - Common anti-pattern
 - WHEN OTHER THEN NULL

```
EXCEPTION
  WHEN OTHERS THEN
    -- Step 1: capture the evidence before it is lost
    INSERT INTO error_log (
      error_code,
      error_message,
      procedure_name,
      logged_at
    ) VALUES (
      SQLCODE,
      SQLERRM,
      'transfer_employee',
      SYSTIMESTAMP
    );
    COMMIT; -- commit the log entry independently of the main transaction

    -- Step 2: tell the caller something went wrong
    RAISE; -- re-raises the original exception unchanged
END;
```


Exception Handling

- Oracle ships a set of named exceptions
 - Common runtime errors
 - Missing rows, duplicate keys, bad conversions, null collections, division by zero, etc
 - Consistent contract because the same logical error always raises the same named exception, regardless of which statement triggered it
- Made up of three parts
 - A name declared in the STANDARD package
 - e.g., NO_DATA_FOUND, DUP_VAL_ON_INDEX, ZERO_DIVIDE)
 - An ORA error code
 - Negative integer returned by SQLCODE
 - The "ORA-01403" form you see in error messages is just the absolute value, zero-padded
 - An error message returned by SQLERRM, which Oracle formats from the code

```
-- Oracle provides named exceptions for the most common error conditions
WHEN NO_DATA_FOUND THEN      -- SELECT INTO returned zero rows
WHEN TOO_MANY_ROWS THEN      -- SELECT INTO returned more than one row
WHEN DUP_VAL_ON_INDEX THEN    -- INSERT or UPDATE violated a unique constraint
WHEN VALUE_ERROR THEN         -- Arithmetic, conversion, or size error
WHEN ZERO_DIVIDE THEN         -- Division by zero
WHEN OTHERS THEN              -- Any exception not otherwise handled
```

Questions?

